

## ЛАБОРАТОРНАЯ РАБОТА № 11

### Тема «Организация ветвлений и циклов в программах»

**Цель:** ознакомиться с синтаксисом условных и циклических операторов, проверить правильность выполнения сценариев, самостоятельно разработать сценарии, содержащие ветвления и циклы.

Время выполнения: 2 часа.

### Выполнить лабораторную работу без использования ИИ.

#### 1 Условный оператор

При составлении программы часто необходимо выполнение различных действий в зависимости от результатов проверки некоторых условий. Для организации ветвлений можно воспользоваться условным оператором, который имеет вид:

```
if B {S1}  
else {S2}
```

где В – выражение логического типа; S1 и S2 – операторы. Выполнение условного оператора осуществляется следующим образом. Вычисляется значение выражения В. Если оно истинно, то выполняются операторы S1, если ложно – операторы S2. Если последовательность операторов S1 или S2 состоит лишь из одного оператора, то фигурные скобки можно опустить. Возможна сокращенная форма условного оператора:

```
if B {S}
```

где В – выражение логического типа; S – последовательность операторов. Выполнение краткого условного оператора осуществляется так: вычисляется значение выражения В, если оно истинно, то выполняются операторы S.

#### 1.1 Максимальное значение

Для трех заданных значений а, b, с необходимо написать сценарий, определяющий максимальное значение.

Поступим следующим образом. Сначала максимальным значением m будем считать а, далее значение b сравним с максимальным. Если окажется, что b больше m, то максимальным становится b. И, наконец, значение c сравнивается с максимальным значением из предыдущих значений а и b. Если c больше m, то максимальным становится c. Оператор присваивания obj.res.value=m обеспечивает запись вычисленного максимального значения в соответствующее поле формы. Функция Number(s) преобразует объект S, заданный в качестве параметра, в число. Полностью сценарий может быть записан так, как представлено в листинге 1.1.

#### Листинг 1.1 – Вычисление максимального значения из трех заданных

```
<!DOCTYPE html>  
<html>  
<head>  
<title>Вычисление максимального значения</title>  
<script>  
function maxval (obj)  
{ var a=Number(obj.num1.value);  
  var b=Number(obj.num2.value);  
  var c=Number(obj.num3.value);
```

```

var m=a
if (b>m) m=b
if (c>m) m=c
obj.res.value=m
}
</script>
</head>
<body>
<h4> Вычисление максимального значения</h4>
<form name="form1">
Число1: <input type="text" size="8" name="num1"><hr>
Число2: <input type="text" size="8" name="num2"><hr>
Число3: <input type="text" size="8" name="num3"><hr>
Максимальное значение равно
<input type="button" value=Определить onClick="maxval(form1)">
<input type="text" size="8" name="res"><hr>
<input type="reset">
</form>
</body>
</html>

```

На рисунке 1 приведен вид документа после введения данных и выполнения сценария, определяющего максимальное значение из введенных.

**Вычисление максимального значения**

Число1:

---

Число2:

---

Число3:

---

Максимальное значение равно

---

Рисунок 1 – Максимальное значение последовательности из трех элементов

Решим рассмотренную задачу другим способом. Стандартный объект Math имеет метод max, который определяет наибольшее значение двух аргументов. Опишем функцию maxval1, которая определяет максимальное значение из трех заданных значений и использует объект Math.

```

function maxval1(obj)
{ var a=Number(obj.num1.value);
  var b=Number(obj.num2.value);
  var c=Number(obj.num3.value);
  obj.res.value=Math.max(Math.max (a,b) , c)
}

```

Если бы требовалось определить максимальное из четырех заданных значений a, b, c, d, то можно было бы воспользоваться формулой

```
Math.max(Math.max (a,b) , Math.max (c,d) )
```

Для четырех заданных значений a, b, c, d необходимо написать сценарий, определяющий максимальное значение.

## 2 Оператор switch и его свойства

В тех случаях, когда при решении задачи требуется выбрать один из вариантов действий из нескольких возможных, удобно воспользоваться оператором switch.

Синтаксис оператора switch следующий:

```
switch (B)
{ case L1: S1;
  case L2: S2;
  ...
  case Ln: Sn;
default: S
}
```

где B – выражение; L1, L2,...,Ln – литералы; S1; S2; ..., Sn; S – операторы.

Выполнение переключателя происходит так: вычисляется значение выражения B. Если значение B равно L1, то выполняются операторы S1, а затем все остальные операторы либо до первого break, либо до конца оператора switch и т.д. Если же значение B не равно ни одному из значений L1, L2, ..., Ln, то выполняются операторы S. Часть default: S может отсутствовать, тогда переключатель имеет вид

```
switch (B)
{ case L1: S1;
  case L2: S2;
  ...
  case Ln: Sn;
}
```

В этом случае, если значение выражения B не равно ни одному из значений L1, L2, ..., Ln, то оператор switch завершает свою работу, что эквивалентно пустому оператору. На самом деле переключатель удобно записывать в виде:

```
switch (B)
{ case L1: S1; break
  case L2: S2; break
  ...
  case Ln: Sn; break
default: S
}
```

В этом случае оператор break обеспечивает завершение работы переключателя после выполнения очередного варианта.

### 2.1 День недели

Напишем программу, которая по номеру дня определяет его название.

Задачу можно решить с использованием условных операторов, в этом случае программа будет ненаглядной. При решении такого рода задач удобно использовать переключатель. Переменной s в зависимости от номера дня недели будет присвоено его название. В листинге 2.1 приведен требуемый сценарий.

#### Листинг 2.1. Определение по номеру дня его названия

```
<!DOCTYPE html>
<html>
<head>
<title>Определение по номеру дня его названия</title>
<script>
function numday (obj)
{ var m=Number(obj.num1.value);
```

```

d=document
var s
switch (m)
{ case 1: s="понедельник"; break;
  case 2: s="вторник"; break;
  case 3: s="среда"; break;
  case 4: s="четверг"; break;
  case 5: s="пятница"; break;
  case 6: s="суббота"; break;
  case 7: s="воскресенье"; break;
  default:s= "ошибка в номере дня"
}
obj.res.value=s
}
</script>
</head>
<body>
<p>Определение названия дня по его номеру</p>
<form name="form1">
Введите номер дня: <input type="text" size="7" name="num1"> <hr>
Название дня:
<input type="button" value="Определить" onClick="numday(form1)">
<input type="text" size="20" name="res"><hr>
<input type="reset">
</form>
</body>
</html>

```

### 3 Итерационные циклы

Для успешного решения круга задач требуется многократно повторять некоторую последовательность действий, записанную в программе один раз. В этом случае, когда число повторений последовательности действий нам неизвестно, либо число повторений (итераций) зависит от некоторых условий, можно воспользоваться оператором цикла с предусловием:

```
while (B) {S}
```

где B – выражение логического типа; S – операторы, называемые телом цикла. Операторы S в фигурных скобках выполняются до тех пор, пока условие B не станет ложным.

Итерационный оператор цикла с постусловием имеет вид:

```
do {S}
```

```
while (B)
```

Этот цикл гарантирует, что операторы будут выполнены хотя бы один раз независимо от того, выполняется ли условие.

#### 3.1 Нахождение общего делителя

Напишем программу, которая для двух заданных чисел определяет наибольший общий делитель.

При решении задачи воспользуемся алгоритмом Евклида. Если значение m равно нулю, то наибольший общий делитель чисел n и m равен n:

НОД (n, m)=НОД (m, n%m).

В функции pod переменная p используется для получения остатка от деления чисел n и m. Выполнение цикла продолжается до тех пор, пока значение p не станет равным нулю. Последнее вычисленное значение m равно наибольшему общему делителю.

Сценарий нахождения общего делителя приведен в листинге 3.1.

## Листинг 3.1 Наибольший общий делитель двух чисел

```

<!DOCTYPE html>
<html>
<head>
<title>Наибольший общий делитель двух чисел</title>
<script>
function nod (obj)
{var n=obj.num1.value
    m=obj.num2.value
    var p=n%m
    while (p!=0)
    {
        n=m
        m=p
        p=n%m
    }
    obj.res.value=m
}
</script>
</head>
<body>
Наибольший общий делитель двух заданных чисел
<form name="form1">
Введите число <input type="text" name="num1" size="8"><br>
Введите число <input type="text" name="num2" size="8"><br>
<input type="button" value="Вычислить" onClick="nod(form1)"><br>
Наибольший общий делитель <input type="text" name="res"
size="8"><hr>
<input type="reset" value="Отменить">
</form>
</body>
</html>

```

Результат выполнения сценария для заданных натуральных чисел представлен на рисунке 2.

Наибольший общий делитель двух заданных чисел

Введите число

Введите число

Наибольший общий делитель

---

Рисунок 2 – Наибольший общий делитель двух натуральных чисел.

На основе сценария нахождения наибольшего общего делителя двух чисел разработайте сценарий нахождения наибольшего общего делителя трех чисел.

Разработайте сценарий нахождения наименьшего общего кратного трех чисел.

Разработайте алгоритм нахождения наибольшего общего делителя, отличный от представленного выше

## 4 Оператор цикла арифметического типа

Если число повторений заранее известно, то можно воспользоваться следующим оператором цикла, который часто называют оператором цикла арифметического типа. Синтаксис этого оператора таков:

```
for (A; B; I) {S}
```

Выражение A служит для инициализации параметра цикла и вычисляется один раз в начале выполнения цикла. Выражение B (условие продолжения) управляет работой цикла. Если значение выражения ложно, то выполнение цикла завершается, если истинно, то выполняется оператор S, составляющий тело цикла. После выполнения тела цикла S вычисляется значение выражения I, затем опять вычисляется значение выражения B и т. д. Цикл может прекратить свою работу в результате выполнения оператора break в теле цикла. Опишем функцию sumdel, которая находит сумму делителей числа n, не считая самого числа.

```
function sumdel (n)
{var s=1;
for (var i=2; i<=n/2; i++)
{ if (n%i==0) s+=1}
return s
}
```

Параметр цикла i описывается с помощью оператора var i=2. Условие продолжения выполнения цикла i<=n/2. Все делители натуральных чисел находятся в интервале [1;n]. Параметр цикла i увеличивается на 1 при выполнении i++. Тело цикла состоит из условного оператора. Если очередное число является делителем, то оно добавляется к переменной s, служащей для «накапливания» суммы делителей числа n. До цикла переменной s присваивается значение 1, так как 1 – делитель любого натурального числа.

### 4.1 Совершенные числа

Напишем программу, определяющую, является ли заданное число n совершенным.

*Совершенным* называется число n, равное сумме своих делителей, не считая самого числа. Например, число 6 является совершенным, т.к. верно  $6=1+2+3$ , где 1, 2, 3 – делители числа 6. Число 28 также является совершенным, справедливо равенство  $28=1+2+4+7+14$ . При решении задачи воспользуемся описанной выше функцией samdel.

Сценарий определения совершенных чисел приведен в листинге 4.1.

#### Листинг 4.1 Совершенные числа

```
<!DOCTYPE html>
<html>
<head>
<title> Совершенные числа</title>
<script>
function sumdel (n)
{var s=1;
for (var i=2; i<=n/2; i++)
{ if (n%i==0) s+=i}
return s
}
function sov (obj)
{ var n=obj.numb.value;
  var s=""
  if (n==sumdel (n)) s="совершенное"
```

```

    else s="не является совершенным"
    return s
  }
</script>
</head>
<body>
<p> Совершенные числа</p>
<form name="form0">
Введите натуральное число: <input type="text" size="8" name="numb">
<input type="button" value="Выполнить"
onClick="this.form.res.value=sov(form0)"><hr>
Данное число: <input type="text" size="24" name="res"><hr>
<input type="reset" value="Отменить">
</form>
</body>
</html>

```

Обратите внимание на значение параметра обработки события. В данном случае это оператор присваивания, в правой части которого вызов функции sov.

Ключевое слово `this` используется при обращении к текущему объекту. В общем случае, с помощью `this` происходит обращение к вызываемому объекту в методах.

Формат:

```
this[.propertyName]
```

Измените сценарий нахождения совершенных чисел таким образом, чтобы ключевое слово `this` использовалось для передачи объекта как параметра.

## 5 Оператор for...in

Оператор `for...in` используется для анализа свойств объекта. Синтаксис оператора:

```
for (i in t) {S}
```

где `i` – переменная цикла; `t` – объект; `S` – последовательность операторов.

В результате выполнения оператора цикла производится перебор свойств объекта. переменная цикла при каждом повторении содержит значение свойств объекта. Количество повторений тела цикла `S` равно числу свойств, определенных для `t`.

### 5.1 Определение свойств элемента формы

Напишем сценарий, с помощью которого можно определить свойства элемента формы «поле ввода многострочного текста».

Сценарий определения свойств текстового поля приведен в листинге 5.1.

Листинг 5.1 Операции над объектами. Свойства текстового поля

```

<!DOCTYPE html>
<html>
<head>
<title>Операции над объектами. Свойства текстового поля</title>
<script>
function propobj (obj)
{var result=""
for (var i in obj)
{result +=obj.data.value+"."+obj.data[i)+"\r\n"}

```

```

result+="\n\r"
form1.data.value=result
}
</script>
</head>
<body bgcolor=f8f8ff>
<center>
<h4> Определение свойств объектов</h4>
<form name="form1">
<input type="button" value="Выполнить" onClick="propobj
(form1)"><hr>
<textarea name="data" cols="30" rows="10" id=1>Текст</textarea><hr>
<input type="reset" value="очистить">
</form>
</center>
</body>
</html>

```

Свойства объекта с помощью оператора цикла формируются в строке result, затем после просмотра всех свойств значение строки result помещается в поле ввода многострочного текста. Результаты сценария представлены на рисунке 3.

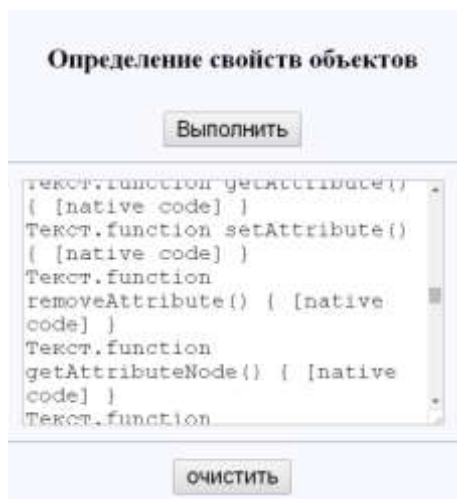


Рисунок 3 – Свойства поля для ввода многострочного текста

На основе сценария определения свойств текстового поля разработайте сценарий определения свойств любого другого элемента формы.

**Уметь самостоятельно (без использования ИИ) составлять рассмотренные в лабораторной работе программы.**



## Самостоятельная работа

### Составить сценарий без использования ИИ.

Требуется спросить у пользователя:

- фамилию, имя, отчество РАЗДЕЛЬНО (три оператора `prompt`)
- возраст в годах (оператором `prompt`)
- пол (оператором `confirm`, например, "ваш пол - мужской?")

и вывести оператором `alert` анкету пользователя по примеру:

```
ваше ФИО: Иванов Иван Иванович  
ваш возраст в годах: 20  
ваш возраст в днях: 7300  
через 5 лет вам будет: 25  
ваш пол: мужской  
вы на пенсии: нет
```

Создать проект ANKETA.

На странице `anketa_if.html` реализовать задание

с использованием условного оператора (`if`), без условной операции (`?:`).

На странице `anketa_oper.html` реализовать задание

с использованием условной операции (`?:`), без условного оператора (`if`).

В обеих реализациях оператор `alert` должен использоваться ровно один раз.

Получить индивидуальное задание у преподавателя. Разобрать условия задач.

Оформить отчет по лабораторной работе с указанием темы, цели работы и кратким описанием выполненных заданий. Электронные документы **безопасно** подключить к сайту.

Защитить лабораторную работу

## ДОМАШНЯЯ РАБОТА

Вдумчиво прочитать учебный материал по учебнику [1, 237 – 242]. Обратить внимание на выполнение оператора `switch`.

Уметь устно отвечать на контрольные вопросы. Письменные ответы записать в тетрадь для лабораторных работ [1, 242].

Разработать сценарии ветвящихся и циклических алгоритмов в соответствии с вариантами индивидуального задания. Выполнить стилевое оформление результирующей веб-страницы.

## ЛИТЕРАТУРНЫЕ ИСТОЧНИКИ

Брылёва, А. А. Программные средства создания интернет-приложений : учеб. Пособие / А. А. Брылёва. – Минск : РИПО, 2022. – 483 с, : и